
Smart Cabin & Vehicle Health Monitoring System

Adaptive Automotive C++ Hackathon
Technical Design & Implementation Document

Team Members

Mohan Ram S Ramya Raman
Harshad Reddy Tannishtha Nair
Anish Singh Chandel

Contents

1	Project Overview	3
1.1	Introduction	3
1.2	Scope and Goals	3
1.3	Real Automotive Mapping	3
2	System Architecture	4
2.1	High-Level Architecture Diagram	4
2.2	Module Dependency Graph	4
3	Sensor Framework	5
3.1	Class Hierarchy	5
3.2	Sensor Data Ranges and Alert Thresholds	5
3.3	Sensor State Machine	6
3.4	Sensor Update Flowchart	7
4	Alert Management Module	8
4.1	Alert Severity Classification	8
4.2	Alert Lifecycle State Machine	8
4.3	Alert Evaluation Flowchart	9
5	Event Logger Module	10
5.1	Async Logging Architecture	10
5.2	Logger State Machine	10
5.3	Log Entry Format	11
6	Threading & Synchronisation	12
6.1	Thread Execution Overview	12
6.2	Thread Synchronisation Model	12
6.3	Startup and Shutdown Flowchart	13
7	Dashboard Module	14
7.1	Dashboard State Machine	14
7.2	Dashboard Refresh Flowchart	15
8	Modern C++ Concepts Used	16
8.1	Concept Coverage Matrix	16
9	Build, Configuration & Usage	17

9.1	Folder Structure	17
9.2	Build Instructions	17
9.3	Configuration Reference (<code>data/config.txt</code>)	18
9.4	Smart Log Viewer (<code>slog</code>)	18
10	Testing Strategy	19
10.1	Test Coverage Overview	19
10.2	Manual Test Scenarios Using Debug Mode	19
11	Automotive Context & Learning Outcomes	20
11.1	Mapping to Adaptive AUTOSAR Concepts	20
A	Key Code Snippets	21
A.1	Abstract Sensor Interface	21
A.2	Thread-Safe Queue Template	21
A.3	Alert Evaluation with Lambda Filtering	21
A.4	RAII Mutex Guard Pattern	22

1. Project Overview

1.1 Introduction

The **Smart Cabin & Vehicle Health Monitoring System** is a production-grade, multi-threaded C++ application that simulates and monitors a vehicle's intelligent cockpit domain controller. It is implemented entirely in C++17 without any external libraries, adhering strictly to modern C++ best practices: RAII, smart pointers, concurrent execution, and STL-centric design.

The system is architected with an *Adaptive AUTOSAR* mindset — featuring service-oriented modules, thread-safe shared data, and event-driven alerting — making it directly analogous to real-world automotive ECU software.

1.2 Scope and Goals

- Simulate six real-world vehicle sensors with periodic data generation.
- Detect abnormal vehicle conditions and classify alerts by severity.
- Present a live, auto-refreshing ASCII dashboard in the terminal.
- Log all critical events asynchronously to a persistent file.
- Manage all concurrent activities safely using `std::thread` and `std::mutex`.
- Provide a manual input debug mode for controlled testing.

1.3 Real Automotive Mapping

System Component	Automotive Equivalent
Sensor Framework	MCAL / Sensor Abstraction Layer
Alert Manager	Vehicle Health Manager (VHM)
Dashboard	Instrument Cluster / HMI
Event Logger	Diagnostic Event Manager (DEM)
Thread Manager	Adaptive AUTOSAR Execution Context
Shared Data Guard	Inter-process Synchronisation
Runtime Alerts	ISO 26262 Safety Monitoring

2. System Architecture

2.1 High-Level Architecture Diagram

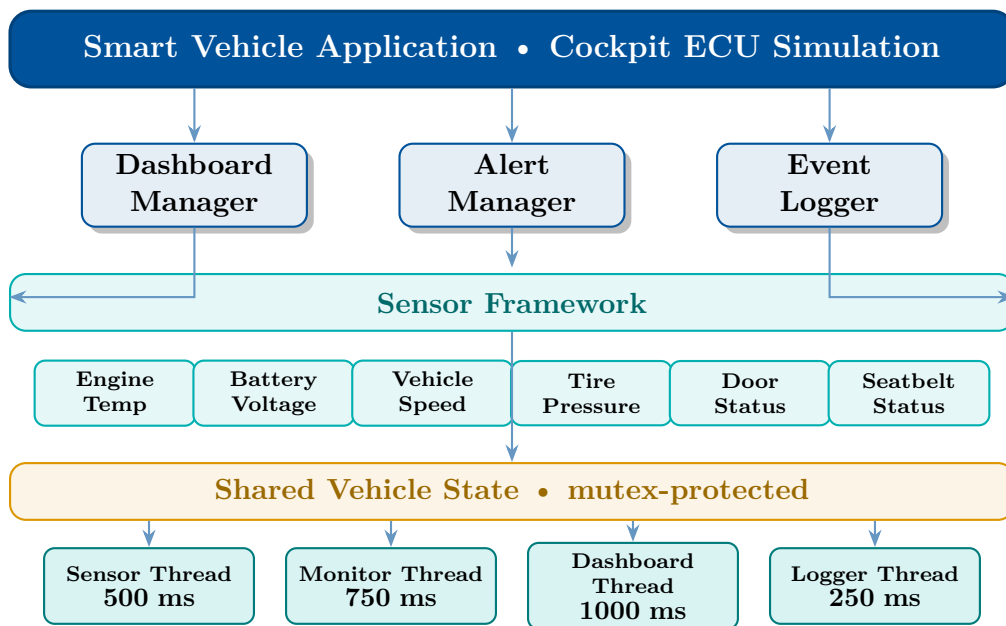


Figure 2.1: High-Level Software Architecture

2.2 Module Dependency Graph

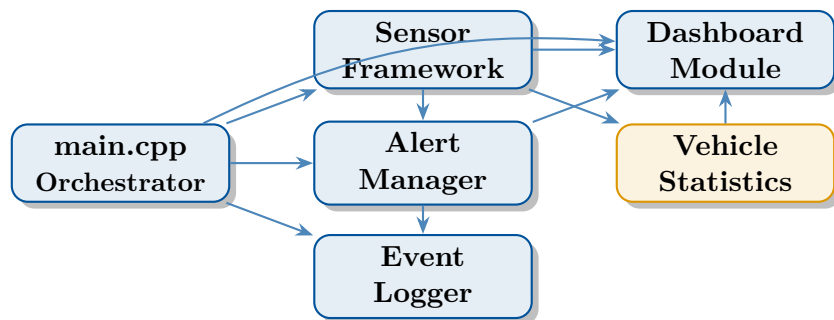


Figure 2.2: Module Dependency Graph

3. Sensor Framework

3.1 Class Hierarchy

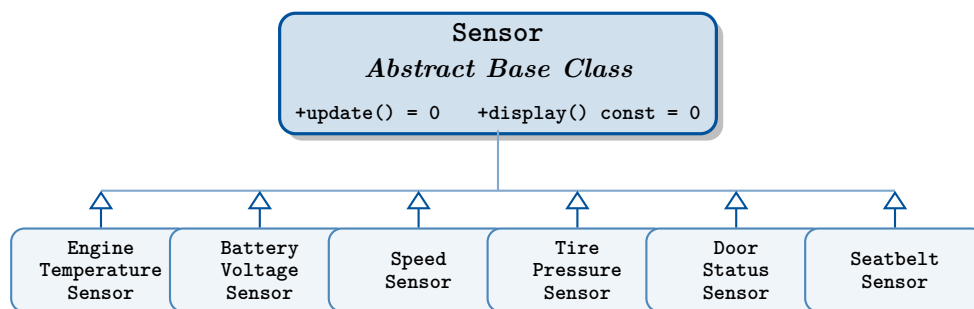


Figure 3.1: Sensor Class Hierarchy (UML Inheritance Tree)

3.2 Sensor Data Ranges and Alert Thresholds

Sensor	Range	Unit	Alert Threshold
Engine Temperature	70 – 120	°C	> 110°C CRITICAL
Battery Voltage	9.5 – 14.8	V	< 10 V WARNING
Vehicle Speed	0 – 220	km/h	> 120 km/h WARNING
Tire Pressure	20 – 40	PSI	< 25 PSI WARNING
Door Status	Open/Closed	—	Open + Speed > 10 km/h CRITICAL
Seatbelt	Locked/Unlocked	—	Unlocked while moving WARNING

3.3 Sensor State Machine

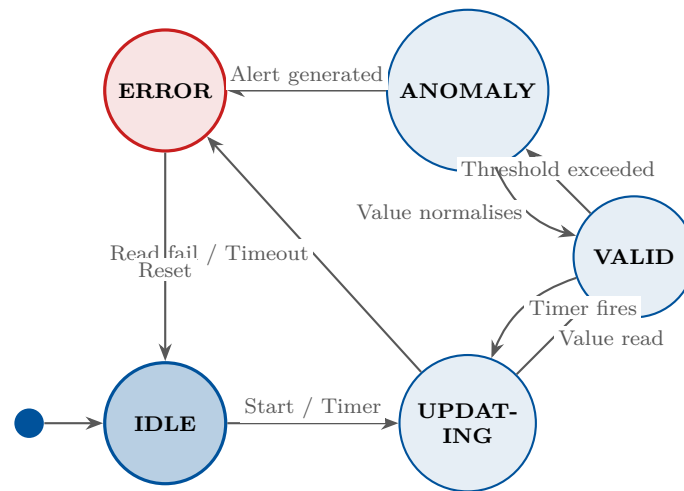


Figure 3.2: Sensor Lifecycle State Machine

3.4 Sensor Update Flowchart

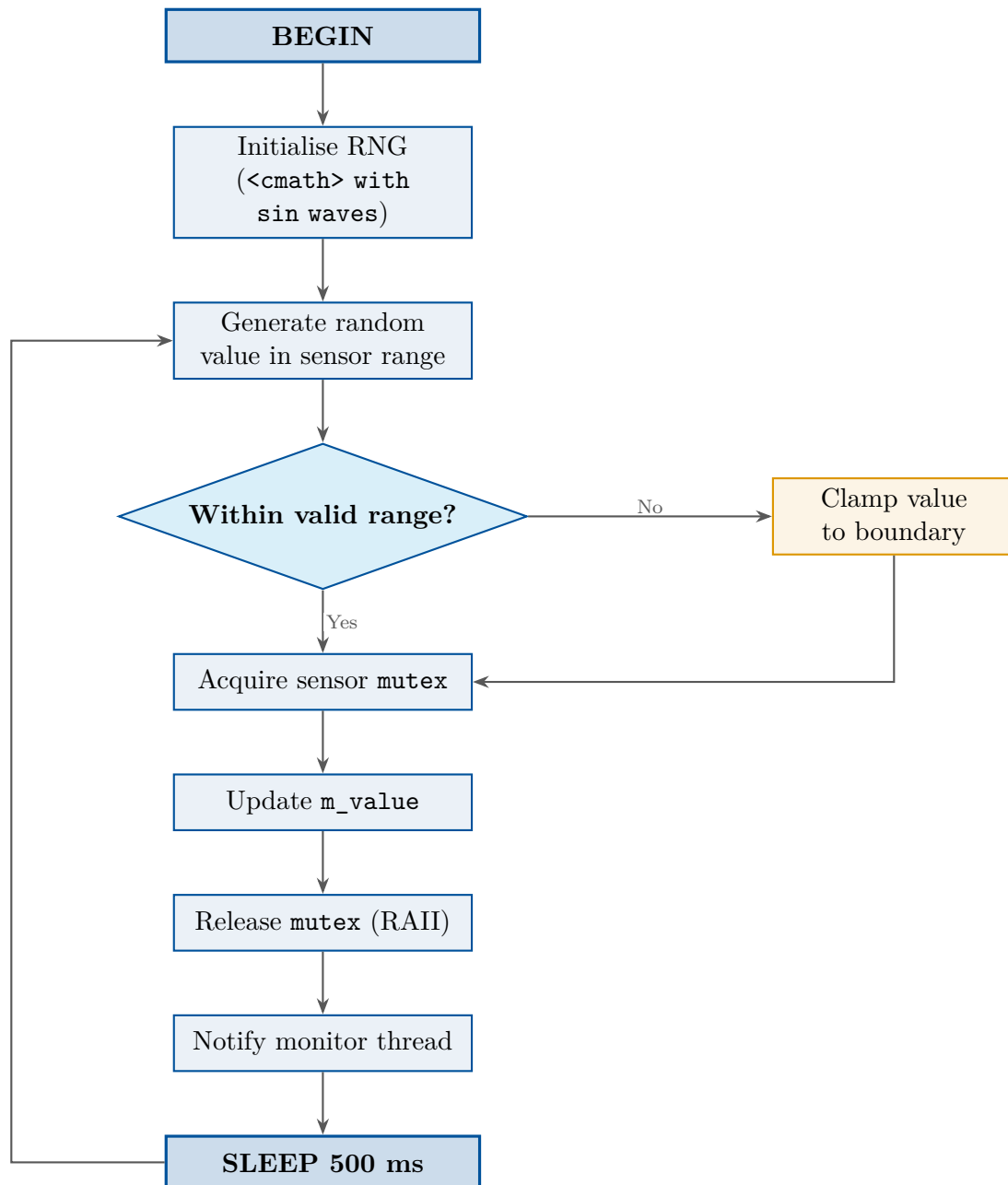


Figure 3.3: Sensor Update Thread Flowchart

4. Alert Management Module

4.1 Alert Severity Classification

Severity	Enum Value	Conditions
INFO	AlertSeverity::INFO	Informational, non-critical events
WARNING	AlertSeverity::WARNING	Battery low, overspeed, tire pressure, seatbelt
CRITICAL	AlertSeverity::CRITICAL	Engine overheat, door open while moving

4.2 Alert Lifecycle State Machine

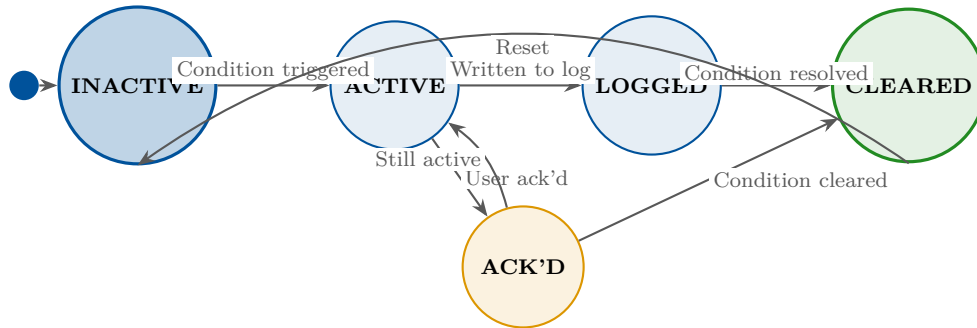


Figure 4.1: Alert Lifecycle State Machine

4.3 Alert Evaluation Flowchart

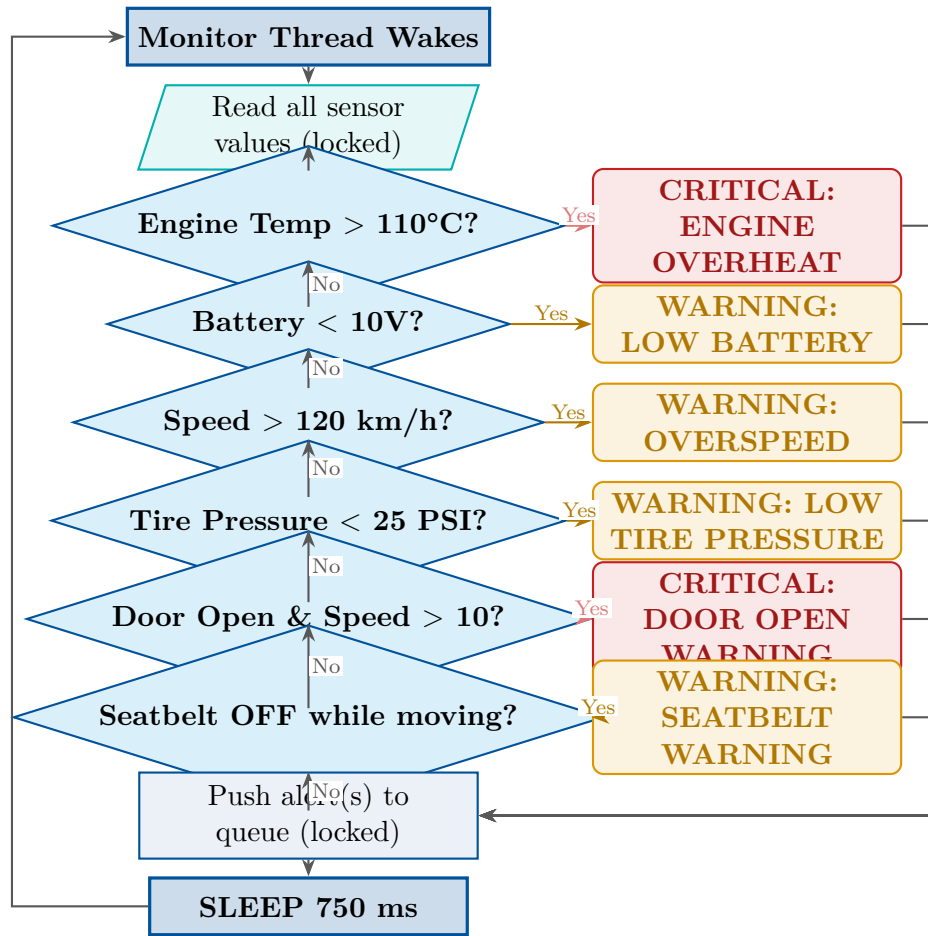


Figure 4.2: Alert Evaluation Flowchart (Monitor Thread)

5. Event Logger Module

5.1 Async Logging Architecture

The Event Logger employs a producer-consumer model using a thread-safe queue. This decouples alert generation from disk I/O, preventing monitor-thread stalls.

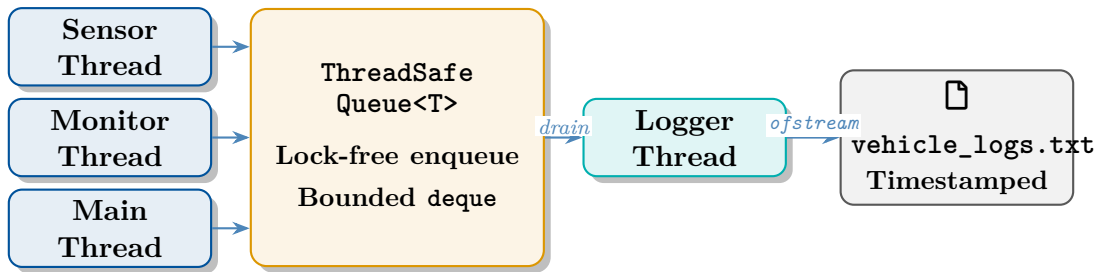


Figure 5.1: Async Event Logger Architecture (Producer-Consumer Pipeline)

5.2 Logger State Machine

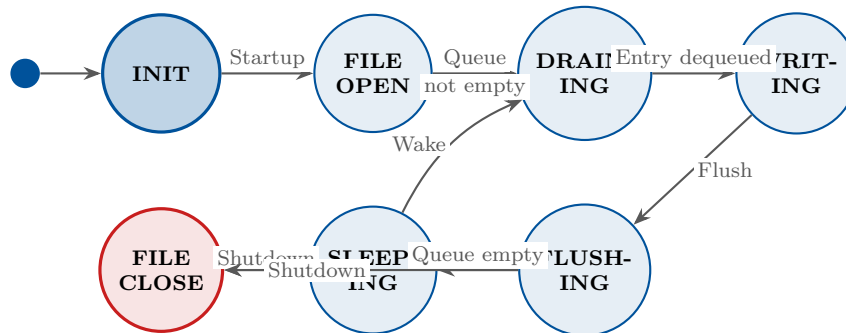


Figure 5.2: Event Logger State Machine

5.3 Log Entry Format

```
1 [2025-06-01 14:32:01] [CRITICAL] ENGINE OVERHEAT | EngineTemp=115.3C
2 [2025-06-01 14:32:03] [WARNING] LOW BATTERY | BatteryV=9.8V
3 [2025-06-01 14:32:07] [WARNING] OVERSPEED | Speed=135.2km/h
4 [2025-06-01 14:32:09] [CRITICAL] DOOR OPEN WARNING | Door=OPEN Speed
   =22.1km/h
5 [2025-06-01 14:32:11] [INFO] System healthy | All sensors nominal
```

Listing 5.1: Sample log output in vehicle_logs.txt

6. Threading & Synchronisation

6.1 Thread Execution Overview

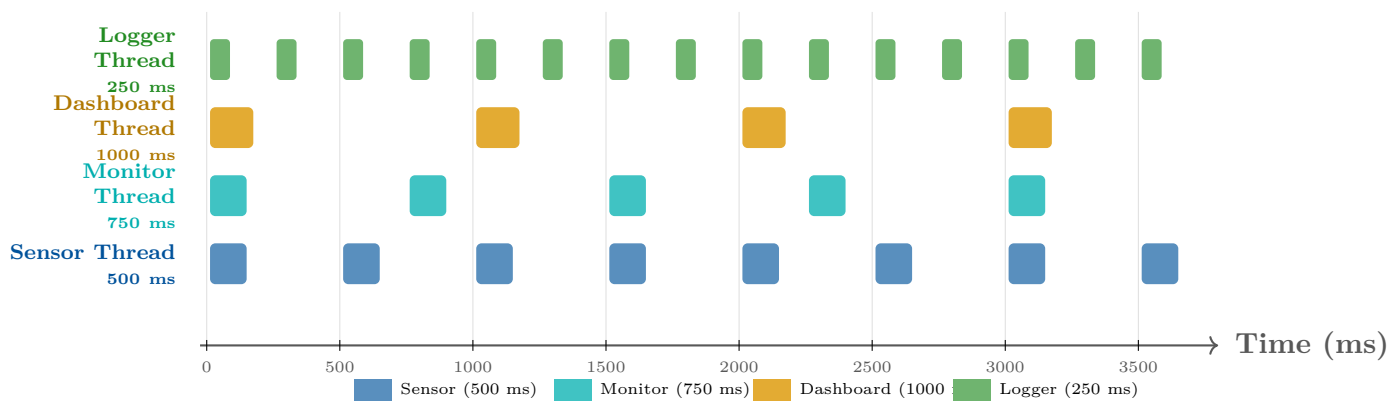


Figure 6.1: Thread Activity Timeline (coloured bursts = execution; gaps = sleep)

6.2 Thread Synchronisation Model

Shared Resource	Readers	Writers	Guard Mechanism
Sensor values	Monitor, Dashboard	Sensor	Per-sensor <code>std::mutex</code>
Active alert list	Dashboard, Logger	Monitor	<code>std::mutex</code> + <code>lock_guard</code>
Alert history	Dashboard	Monitor	Same alert mutex
Log queue	Logger thread	All	<code>ThreadSafeQueue</code> internal mutex
Statistics map	Dashboard	Sensor	Statistics mutex
Config flags	All	Main	Atomic read, start-up only

6.3 Startup and Shutdown Flowchart

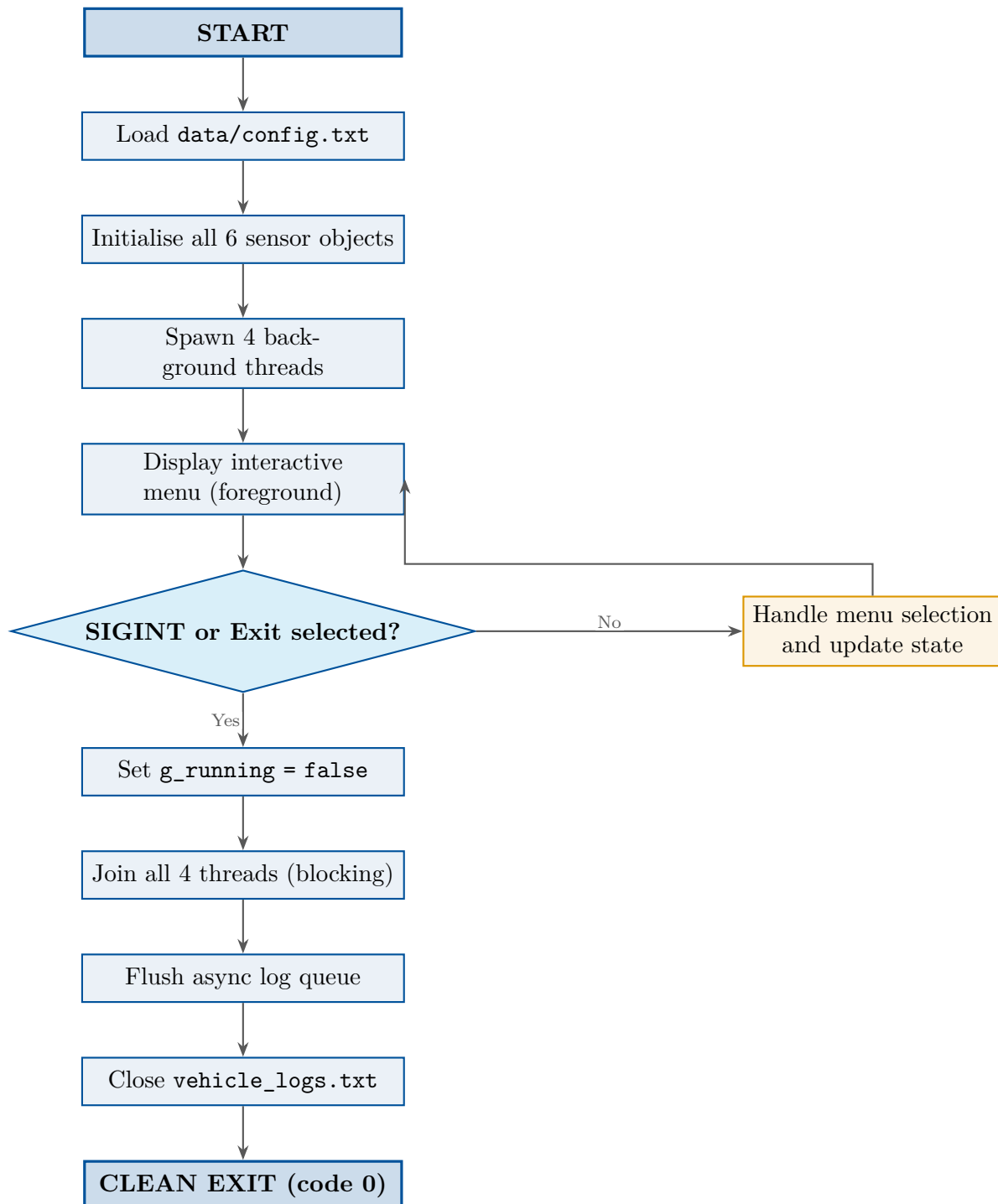


Figure 6.2: Application Startup and Graceful Shutdown Flowchart

7. Dashboard Module

7.1 Dashboard State Machine

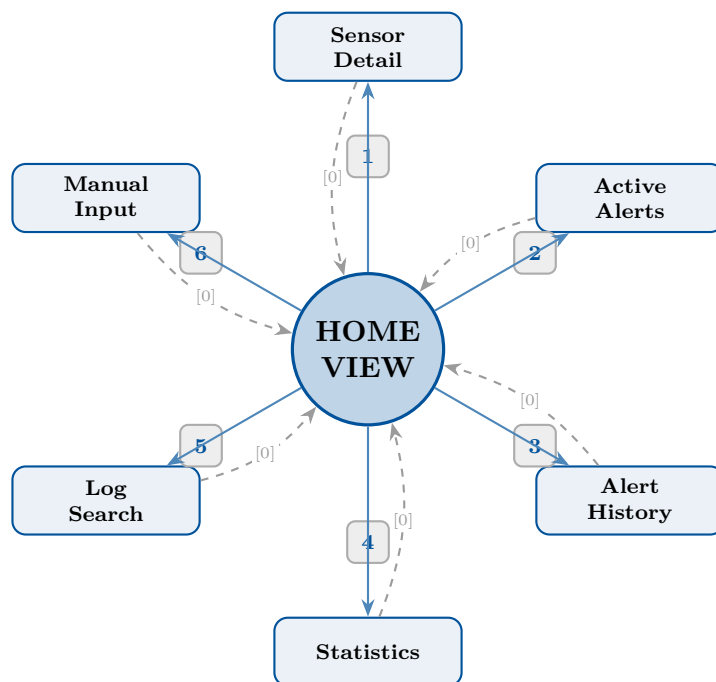


Figure 7.1: Dashboard Navigation State Machine (solid = go, dashed = return to Home)

7.2 Dashboard Refresh Flowchart

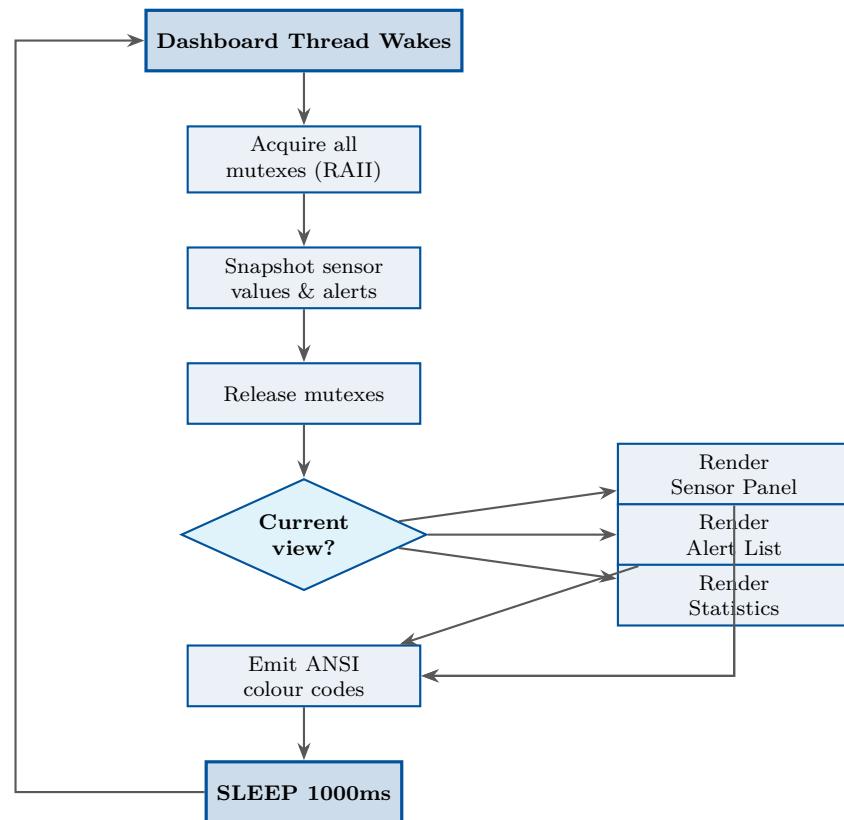


Figure 7.2: Dashboard Refresh Flowchart

8. Modern C++ Concepts Used

8.1 Concept Coverage Matrix

C++ Feature	Where Used	Why
Abstract Base Class / Virtual Functions	Sensor base class	Runtime polymorphism; dispatch to correct <code>update()</code>
RAII	File handles, mutex locks	Guaranteed release; no leaks on exceptions
<code>unique_ptr</code> / <code>shared_ptr</code>	Sensor objects, alert entries	Safe ownership, no raw <code>new/delete</code>
<code>std::thread</code> + <code>std::mutex</code>	All 4 background threads	Concurrent sensor updates without data races
<code>std::lock_guard</code>	Every shared data access	Automatic unlock; exception-safe
<code>enum class</code>	<code>AlertSeverity</code>	Scoped, type-safe severity levels
Operator Overloading (<code><<</code>)	Alert class	Formatted output via stream operators
Lambda Expressions	Log search, alert filtering	Concise predicates for <code>std::copy_if</code>
<code>std::vector</code>, <code>std::deque</code>, <code>std::map</code>	Alert list, history, statistics	Chosen per access-pattern requirements
Templates	<code>ThreadSafeQueue<T></code>	Generic, reusable concurrent queue
<code>std::mt19937</code>	Per-sensor RNG	High-quality pseudo-random sensor simulation
Move Semantics	Alert event objects	Efficient transfer without unnecessary copies
Static Members	Alert counter, object tracking	Class-level bookkeeping
Exception Handling	File open failures, bad data	Robust error handling without crashes

9. Build, Configuration & Usage

9.1 Folder Structure

```
1 cpp_hack/  
2     include/  
3         Sensor.hpp           # Abstract base + derived sensor  
4     headers  
5         Alert.hpp            # Alert class and AlertManager  
6         Logger.hpp           # ThreadSafeQueue and EventLogger  
7         Dashboard.hpp        # Dashboard and VehicleStatistics  
8     src/  
9         main.cpp             # Orchestration, thread launch,  
10    SIGINT handler  
11        Sensor.cpp  
12        Alert.cpp  
13        Logger.cpp  
14        Dashboard.cpp  
15    data/  
16    logs/  
17        config.txt           # Runtime configuration flags  
18        vehicle_logs.txt     # Timestamped event log (auto-created  
19    )  
20    tests/                   # Regression tests  
21        __pycache__/  
22        test_cases.py  
23        test_report.md  
24        test_runner.py  
25    Makefile  
26    slog                     # Bash smart log viewer  
27    README.md
```

Listing 9.1: Repository structure

9.2 Build Instructions

```
1 # Clean previous build artifacts  
2 make clean  
3  
4 # Compile with C++17 (GCC/Clang required)  
5 make  
6  
7 # Run the application  
8 ./vehicle_monitor
```

Listing 9.2: Building and running the application

9.3 Configuration Reference (data/config.txt)

Key	Default	Description
MANUAL_INPUT_MODE	0	Set to 1 to enable manual sensor value injection
SENSOR_INTERVAL_MS	500	Milliseconds between sensor updates
LOG_FILE_PATH	logs/vehicle_logs.txt	Output log file path
MAX_ALERT_HISTORY	100	Maximum entries retained in alert history deque
OVERSPEED_LIMIT	120	Speed threshold (km/h) for overspeed alert
ENGINE_TEMP_LIMIT	110	Temperature threshold (°C) for engine alert
BATTERY_MIN_VOLT	10.0	Voltage floor (V) for low battery alert
TIRE_MIN_PSI	25	Pressure floor (PSI) for tire alert

9.4 Smart Log Viewer (slog)

```

1 chmod +x slog
2
3 ./slog                # Print all log entries
4 ./slog -f             # Tail the log in real-time (tail -f mode)
5 ./slog -n 30          # Show the last 30 log entries
6 ./slog CRITICAL       # Filter entries by severity level
7 ./slog "ENGINE OVERHEAT" # Search for a specific alert phrase
8 ./slog --clear        # Wipe the log file

```

Listing 9.3: slog usage examples

10. Testing Strategy

10.1 Test Coverage Overview

Test Area	Approach	Tool
Sensor range clamping	Regression test; feed boundary values	<code>tests/</code> + assert
Alert condition evaluation	Regression test; mock sensor readings	Manual + <code>MANUAL_INPUT_MODE</code>
Thread safety	Race condition detection	Valgrind / ThreadSanitizer
Log file persistence	Verify entries after clean restart	<code>slog</code> script
Graceful shutdown	SIGINT; verify all threads joined	OS signal + exit code 0
Memory leak detection	Full lifecycle run	Valgrind <code>-leak-check=full</code>

10.2 Manual Test Scenarios Using Debug Mode

1. Set `MANUAL_INPUT_MODE=1` in `config.txt`.
2. Launch `./vehicle_monitor`.
3. Choose [6] **Manual Sensor Input**.
4. Enter engine temperature of 115°C → verify `CRITICAL: ENGINE OVERHEAT` appears.
5. Enter speed 130 km/h and door = OPEN → verify both overspeed and door-open alerts.
6. Enter battery voltage 9.5 V → verify `WARNING: LOW BATTERY`.
7. Restore values to nominal → verify alerts clear from active list.
8. Press Ctrl-C → verify clean exit with no orphan threads.

11. Automotive Context & Learning Outcomes

11.1 Mapping to Adaptive AUTOSAR Concepts

Hackathon Concept	Adaptive AUTOSAR / Industry Equivalent
Sensor abstract base	AUTOSAR Sensor & Actuator Abstraction
AlertManager	Vehicle Health Manager (VHM)
EventLogger	Diagnostic Event Manager (DEM)
Dashboard	HMI / Instrument Cluster IPC
std::thread	Adaptive AUTOSAR Execution Context
std::mutex	Inter-process / inter-runnable synchronisation
g_running atomic flag	Watchdog / lifecycle management
RAII file handles	Resource Manager in safety-critical software
ThreadSafeQueue	AUTOSAR Ara::Com service queuing

A. Key Code Snippets

A.1 Abstract Sensor Interface

```
1 class Sensor {
2 public:
3     virtual void update() = 0;
4     virtual void display() const = 0;
5     virtual std::string getName() const = 0;
6     virtual ~Sensor() = default;
7
8 protected:
9     mutable std::mutex m_mutex;
10    std::mt19937          m_rng{std::random_device{}()};
11 };
```

Listing A.1: Sensor base class (include/Sensor.hpp)

A.2 Thread-Safe Queue Template

```
1 template<typename T>
2 class ThreadSafeQueue {
3 public:
4     void push(T item) {
5         std::lock_guard<std::mutex> lock(m_mutex);
6         m_queue.push_back(std::move(item));
7     }
8     bool pop(T& out) {
9         std::lock_guard<std::mutex> lock(m_mutex);
10        if (m_queue.empty()) return false;
11        out = std::move(m_queue.front());
12        m_queue.pop_front();
13        return true;
14    }
15 private:
16    std::deque<T> m_queue;
17    std::mutex m_mutex;
18 };
```

Listing A.2: Generic lock-based producer-consumer queue

A.3 Alert Evaluation with Lambda Filtering

```
1 auto results = std::vector<LogEntry>{};
2 std::copy_if(
3     m_history.begin(), m_history.end(),
4     std::back_inserter(results),
5     [&keyword](const LogEntry& e) {
6         return e.message.find(keyword) != std::string::npos;
7     }
8 );
```

Listing A.3: Lambda-based log search in EventLogger

A.4 RAII Mutex Guard Pattern

```
1 void MonitorThread::evaluate() {
2     // Snapshot under lock; release before alert processing
3     SensorSnapshot snap;
4     {
5         std::lock_guard<std::mutex> lock(m_sensorMutex);
6         snap = collectSnapshot();
7     } // mutex released here automatically
8
9     if (snap.engineTemp > 110.0f)
10         m_alertManager.raise(AlertSeverity::CRITICAL,
11                               "ENGINE OVERHEAT");
12 }
```

Listing A.4: RAII lock usage in monitor thread